

Task 10: Exception Handling in Python (try-except)

Track: Data Science | Level 1 - Day 10 | Tools: Python, Jupyter Notebook

Level 1 - Day 10: Exception Handling in Python (`try-except`)

Track: Data Science

Task: Task 10 - Robust Error and Exception Handling for Data Pipelines

Objective: Master defensive programming in data science using `try`, `except`, `else`, `finally`, raising custom exceptions, handling common data processing errors (`ValueError`, `TypeError`, `ZeroDivisionError`, `KeyError`, `FileNotFoundError`), and gracefully processing corrupted datasets.

1. Quick Reference: Python Exception Handling Architecture

Block / Keyword	Syntax	Role in Data Preprocessing Pipelines
`try`	`try:`	Encapsulates risky parsing, division, or data extraction code
`except ExceptionType as e:`	`except ValueError as err:`	Catches specific error types, prevents pipeline crashes, logs issue
`else`	`else:`	Executes strictly when no exception occurs in `try`
`finally`	`finally:`	Always executes; used for resource cleanup (closing DB/file streams)
`raise`	`raise ValueError(...)`	Enforces data quality gates and schema validation checks

2. Core Exception Handling Patterns (8 Practical Scenarios)

In [code]

```
# Example 1: Handling ZeroDivisionError during metric calculation
# Calculating conversion rates where impressions may be zero
clicks = 120
impressions = 0

try:
    click_through_rate = clicks / impressions
    print(f"CTR: {click_through_rate:.2%}")
except ZeroDivisionError as e:
    click_through_rate = 0.0
    print(f"Ex 1 - Handled ZeroDivisionError: Zero impressions recorded. Defaulted CTR to {click_through_rate}")
```

In [code]

```
# Example 2: Handling ValueError on numerical casting from dirty strings
# Parsing messy numeric inputs from web scraping
raw_age_input = "28_unknown"

try:
    cleaned_age = int(raw_age_input)
    print(f"Parsed age: {cleaned_age}")
except ValueError as e:
    cleaned_age = None
    print(f"Ex 2 - Handled ValueError on '{raw_age_input}': {e}. Imputed value as None.")
```

In [code]

```
# Example 3: Handling TypeError on incompatible feature operations
# Adding bonus increments where field types are mismatched
base_salary = 75000
bonus_grant = "5000" # String instead of numeric int/float

try:
    total_comp = base_salary + bonus_grant
except TypeError as e:
    # Fallback casting correction
    total_comp = base_salary + float(bonus_grant)
    print(f"Ex 3 - Handled TypeError: {e}. Coerced type dynamically -> Total Comp: ${total_comp:,.2f}")
```

In [code]

```
# Example 4: Handling KeyError on missing schema dictionary fields
# Accessing optional user profile metadata safely
user_record = {"user_id": "U-802", "name": "Priya", "role": "Data Analyst"}

try:
    department = user_record["department"]
except KeyError as e:
    department = "General / Unassigned"
    print(f"Ex 4 - Handled KeyError {e}: Missing optional key. Fallback set to '{department}'")
```

In [code]

```
# Example 5: Handling IndexError on missing sequence elements
# Extracting test split coordinates
coordinate_pairs = [(10.2, 55.4), (20.8, 48.9)]

try:
    target_pair = coordinate_pairs[5]
except IndexError as e:
    target_pair = coordinate_pairs[-1]
    print(f"Ex 5 - Handled IndexError: {e}. Fallback to last recorded coordinate: {target_pair}")
```

In [code]

```
# Example 6: Multiple Exception Clauses in a single try block
# Safely evaluating arbitrary JSON metrics
def calculate_sample_metric(data_dict, key, divisor):
    try:
        val = data_dict[key]
        result = float(val) / divisor
        return result
    except KeyError:
        return "Error: Key Not Found"
    except ZeroDivisionError:
        return "Error: Cannot Divide By Zero"
    except ValueError:
        return "Error: Non-numeric Value"

sample_dict = {"score": "95", "invalid": "N/A"}
print(f"Ex 6a - Valid: {calculate_sample_metric(sample_dict, 'score', 5)}")
print(f"Ex 6b - Key Error: {calculate_sample_metric(sample_dict, 'missing', 5)}")
print(f"Ex 6c - Zero Division: {calculate_sample_metric(sample_dict, 'score', 0)}")
print(f"Ex 6d - Value Error: {calculate_sample_metric(sample_dict, 'invalid', 2)}")
```

In [code]

```
# Example 7: Utilizing the 'else' and 'finally' blocks
# Mock dataset connection and resource teardown
connection_status = "Disconnected"
try:
    connection_status = "Connected to Staging DB"
    # Simulation of query execution
    rows_retrieved = 1500
    print(f"Ex 7 - [try] Query succeeded: fetched {rows_retrieved} records.")
except Exception as err:
    print(f"Ex 7 - [except] Encountered error: {err}")
else:
    print("Ex 7 - [else] Commit transaction and execute downstream cache.")
finally:
    connection_status = "Disconnected (Clean)"
    print(f"Ex 7 - [finally] Cleaned up socket. Connection status: {connection_status}")
```

In [code]

```
# Example 8: Raising Custom Exceptions for Data Quality Gates
# Checking that percentages fall strictly within [0, 100]
def validate_attendance_pct(pct):
    if not isinstance(pct, (int, float)):
        raise TypeError(f"Attendance must be numeric, received {type(pct).__name__}")
    if pct < 0.0 or pct > 100.0:
        raise ValueError(f"Invalid attendance value: {pct}%. Must be between 0 and 100.")
    return True

test_values = [85.5, 115.0, "88"]
for val in test_values:
    try:
        validate_attendance_pct(val)
        print(f"Ex 8 - Value {val}% passed validation.")
    except (ValueError, TypeError) as quality_err:
        print(f"Ex 8 - Validation Alert on '{val}': {quality_err}")
```

3. End-to-End Fault-Tolerant Data Pipelines

Real-world data ingestion pipelines processing corrupted and incomplete **Student Exam** and **Employee Payroll** datasets without halting execution.

In [code]

```
# Pipeline 1: Fault-Tolerant Student Exam Ingestion & Grading Engine
# Dirty mock batch with type errors, nulls, missing keys, and division risks
raw_student_batch = [
    {"id": "ST1", "name": "Aman", "marks": [80, 90, 85], "attendance": 90},
    {"id": "ST2", "name": "Pooja", "marks": ["60", 70, "missing"], "attendance": 75},
    {"id": "ST3", "name": "Rohan", "marks": [], "attendance": 0},
    {"id": "ST4", "name": "Sneha", "marks": [95, 98, 92]}, # Missing attendance key
    {"id": "ST5", "name": "Vikas", "marks": [70, 75, 80], "attendance": 85}
]

processed_students = []
error_audit_log = []

print("=== ROBUST STUDENT INGESTION PIPELINE REPORT ===")
for record in raw_student_batch:
    student_id = record.get("id", "UNKNOWN")
    name = record.get("name", "Unknown")

    # 1. Parse and sanitize marks
    clean_marks = []
    for m in record.get("marks", []):
        try:
            clean_marks.append(float(m))
        except (ValueError, TypeError):
            error_audit_log.append(f"[{student_id}] Non-numeric mark '{m}' skipped.")

    # 2. Compute mean score with ZeroDivisionError guard
    try:
        avg_score = round(sum(clean_marks) / len(clean_marks), 2)
    except ZeroDivisionError:
        avg_score = 0.0
        error_audit_log.append(f"[{student_id}] No valid marks found; average defaulted to 0.0.")

    # 3. Retrieve attendance with KeyError guard
    try:
        att = float(record["attendance"])
    except KeyError:
        att = 60.0 # Cohort median fallback
        error_audit_log.append(f"[{student_id}] Missing 'attendance' key; imputed fallback {att}%.")
    except (ValueError, TypeError):
        att = 60.0
        error_audit_log.append(f"[{student_id}] Corrupt attendance value; imputed fallback {att}%.")

    processed_students.append({
        "id": student_id,
        "name": name,
        "avg_score": avg_score,
        "attendance": att,
        "status": "Distinction" if avg_score >= 80 and att >= 75 else ("Pass" if avg_score >= 50 else
"Fail")
    })

    print(f"ID: {student_id} | Name: {name:<6} | Clean Avg: {avg_score:>5.2f}% | Att: {att:>4.1f}% |
Grade: {processed_students[-1]['status']}")

print("\n--- Ingestion Error Audit Log ---")
for entry in error_audit_log:
    print(f"    • {entry}")
```

In [code]

```
# Pipeline 2: Resilient Corporate Payroll Parser & Budget Reconciler
raw_payroll_feed = [
    {"emp_id": "E101", "name": "Aditi", "dept": "Tech", "salary": "120000", "rating": 4.9},
    {"emp_id": "E102", "name": "Karan", "dept": "Sales", "salary": "NULL", "rating": 3.5},
    {"emp_id": "E103", "name": "Neha", "dept": "Tech", "salary": 85000, "rating": "invalid"},
    {"emp_id": "E104", "name": "Rahul", "dept": "HR", "salary": -4000, "rating": 4.2} # Negative
salary anomaly
]

clean_payroll = []
payroll_errors = []

print("\n=== RESILIENT PAYROLL PARSER & AUDIT ENGINE ===")
for emp in raw_payroll_feed:
    e_id = emp.get("emp_id", "E-UNK")
    name = emp.get("name", "Anonymous")
    dept = emp.get("dept", "General")

    # Validate salary
    try:
        sal = float(emp["salary"])
        if sal <= 0:
            raise ValueError(f"Negative or zero salary recorded ({sal})")
    except (ValueError, TypeError, KeyError) as sal_err:
        sal = 50000.0 # Base company floor salary
        payroll_errors.append(f"[{e_id}] Salary issue ({sal_err}). Imputed base floor ${sal:,.0f}")

    # Validate rating
    try:
        rating = float(emp["rating"])
        if not (1.0 <= rating <= 5.0):
            raise ValueError(f"Rating out of bounds ({rating})")
    except (ValueError, TypeError, KeyError) as rate_err:
        rating = 3.0 # Neutral review default
        payroll_errors.append(f"[{e_id}] Rating issue ({rate_err}). Imputed neutral default 3.0")

    hike = 0.10 if rating >= 4.5 else 0.05
    final_salary = sal * (1.0 + hike)

    clean_payroll.append({"id": e_id, "name": name, "dept": dept, "salary": final_salary})
    print(f"[{e_id}] {name:<6} ({dept:<5}) | Base: ${sal:>9,.2f} | Rating: {rating:.1f} | Final: $
{final_salary:>10,.2f}")

print("\n--- Payroll Audit Anomaly Log ---")
for err in payroll_errors:
    print(f" • {err}")
```